

Due: Tuesday, April 2 at 11:59 pm

Submit your predictions for the test sets to Kaggle as early as possible. Include your Kaggle scores in your write-up (see below). The Kaggle competition for this assignment can be found at

- Spam: <https://www.kaggle.com/competitions/cs189-hw5-spam-spring-2024>
- Titanic: <https://www.kaggle.com/competitions/cs189-hw5-titanic-spring-2024>

Write-up: Submit your solution in **PDF** format to “Homework 5 Write-Up” on Gradescope.

- State your name, and if you have discussed this homework with anyone (other than GSIs), list the names *of them all*.
- Begin the solution for each question in a new page. Do not put content for different questions in the same page. You may use multiple pages for a question if required.
- If you include figures, graphs or tables for a question, any explanations should accompany them in *the same page*. Do NOT put these in an appendix!
- **Only PDF uploads to Gradescope will be accepted.** You may use L^AT_EX or Word to typeset your solution or scan a neatly handwritten solution to produce the PDF.
- **Replicate all your code in an appendix.** Begin code for each coding question in a fresh page. Do not put code from multiple questions in the same page. When you upload this PDF on Gradescope, *make sure* that you assign the relevant pages of your code from appendix to correct questions.

Code: Additionally, submit all your code as a .zip file to “Homework 5 Code” on Gradescope.

- **Set a seed for all pseudo-random numbers generated in your code.** This ensures your results are replicated when readers run your code.
- Include a README with your name, student ID, the values of the random seed (above) you used, and instructions for running (and compiling, if appropriate) your code.
- Do NOT provide any data files, but supply instructions on how to add data to your code.
- Code that the readers can’t run because it requires exorbitant memory or execution time might not receive marks.
- Code submitted here must match that in the PDF Write-up, and produce the *exact* output submitted to Kaggle. Inconsistent or incomplete code might not receive marks.

1 Honor Code

Declare and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature : _____

While discussions are encouraged, *everything* in your solution must be your (and only your) creation. Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

2 Random Forest Motivation

Ensemble learning is a general technique to combat overfitting, by combining the predictions of many varied models into a single prediction based on their average or majority vote.

- (a) **The motivation of averaging.** Consider a set of uncorrelated random variables $\{Y_i\}_{i=1}^n$ with mean μ and variance σ^2 . Calculate the expectation and variance of their average. (In the context of ensemble methods, these Y_i ’s are analogous to the prediction made by classifier i .)
- (b) In part (a), we see that averaging reduces variance for uncorrelated classifiers. Real-world prediction will of course not be completely uncorrelated, but reducing correlation among decision trees will generally reduce the final variance. Reconsider a set of correlated random variables $\{Z_i\}_{i=1}^n$ with mean μ and variance σ^2 , where each $Z_i \in \mathbb{R}$ is a scalar. Suppose $\forall i \neq j$, $\text{Corr}(Z_i, Z_j) = \rho$. (If you don’t remember the relationship between correlation and covariance from your prerequisite classes, please look it up.) Calculate the variance of the average of the random variables Z_i , written in terms of σ , ρ , and n .

What happens when n gets very large, and what does that tell us about the potential effectiveness of averaging? (... if ρ is large? ... if ρ is very very small? ... if ρ is middling?)

- (c) **Ensemble Learning – Bagging.** In lecture, we covered bagging (Bootstrap AGGREGatING). Bagging is a randomized method for creating many different learners from the same data set.

Given a training set of size n , generate T random subsamples, each of size n' , by sampling with replacement. Some points may be chosen multiple times, while some may not be chosen at all. If $n' = n$, around 63% are chosen, and the remaining 37% are called out-of-bag (OOB) sample points.

- (i) Why 63%?

*Hint: when n is very large, what is the probability that a sample point won’t be selected? Please only consider the probability of a point not being selected in any **one** of the subsamples (not all of the T subsamples).*

- (ii) The number of decision trees T in the ensemble is usually chosen to trade off running time against reduced variance. (Typically, a dozen to several thousand trees are used.) The sample size n' has a smaller effect on running time, so our choice of n' is mainly governed by getting the best predictions. Although it’s common practice to set $n' = n$, that isn’t necessarily the best choice. How do you recommend we choose the hyperparameter n' ?

$$(a) \quad E\left[\frac{1}{n} \sum_{i=1}^n \{Y_i\}\right] = \frac{1}{n} \sum_{i=1}^n E[Y_i] = \frac{1}{n} \cdot n \cdot \mu = \mu$$

↖ $\{Y_i\}_{i=1}^n$ are uncorrelated

$$\text{Var}\left[\frac{1}{n} \sum_{i=1}^n \{Y_i\}\right] = \frac{1}{n^2} \text{Var}\left[\sum_{i=1}^n \{Y_i\}\right] = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(Y_i) = \frac{1}{n^2} \cdot n \cdot \sigma^2 = \frac{\sigma^2}{n}$$

Averaging reduces the variance from σ^2 to σ^2/n

$$(b) \quad \text{Var}\left[\frac{1}{n} \sum_{i=1}^n \{z_i\}\right] = E\left[\left(\frac{1}{n} \sum_{i=1}^n \{z_i\}\right)^2\right] - \left(E\left[\frac{1}{n} \sum_{i=1}^n \{z_i\}\right]\right)^2$$

$$= E\left[\frac{1}{n^2} \left(\sum_{i=1}^n \{z_i\} \cdot \sum_{j=1}^n \{z_j\}\right)\right] - \mu^2$$

$$= \frac{1}{n^2} E\left[\sum_{i=1}^n \sum_{j=1}^n z_i \cdot z_j\right] - \mu^2$$

$$E\left[\sum_{i=1}^n \sum_{j=1}^n z_i z_j\right] = \sum_{i=1}^n \sum_{j=1}^n \text{Cov}(z_i, z_j) + E\left[\sum_{i=1}^n z_i\right] \cdot E\left[\sum_{j=1}^n z_j\right]$$

$$= \sum_{i=1}^n \sum_{j=1}^n \text{Cov}(z_i, z_j) + n^2 \cdot \mu^2$$

$$\sum_{i=1}^n \sum_{j=1}^n \text{Cov}(z_i, z_j) = \sum_{i=1}^n \text{Var}[z_i] + \sum_{i \neq j} \text{Corr}(z_i, z_j) \cdot \sigma^2$$

$$= n \cdot \sigma^2 + n(n-1) \cdot \rho \cdot \sigma^2$$

$$\Rightarrow \text{Var}\left[\frac{1}{n} \sum_{i=1}^n \{z_i\}\right] = \frac{1}{n^2} [n\sigma^2 + n(n-1)\rho \cdot \sigma^2]$$

$$= \frac{1 + (n-1)\rho}{n} \cdot \sigma^2$$

when n is very large $\frac{1 + (n-1)\rho}{n} \cdot \sigma^2 \rightarrow \rho \sigma^2$

$$0 \leq \rho \leq 1$$

if $\rho = 1$ completely correlated : averaging does not reduce variance

if $\rho \sim 0$ completely uncorrelated : averaging is very effective at reducing variance

if $0 < \rho < 1$: averaging reduces variance to some extent.

(c) (i) if $n' = n$

one point in n -samples not chosen in n' -size subsamples

note that drawing with replacement

$$\underbrace{\frac{n-1}{n} \cdot \frac{n-1}{n} \cdots \frac{n-1}{n}}_{n' \text{ times, namely } n \text{ times}} = \left(\frac{n-1}{n}\right)^n = \left(1 - \frac{1}{n}\right)^n$$

n' times, namely n times

when n is very large

$$\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = \frac{1}{e}$$

which means the probability of the point in n -samples

chosen in n' -size subsamples is

$$1 - \frac{1}{e} \approx 63.2\%$$

$$E\left[\frac{1}{n} \sum_{i=1}^n \underset{\uparrow}{p_i}\right] = \frac{1}{n} \sum_{i=1}^n E[p_i] = \bar{p} = 63.2\%$$

$\uparrow$ point chosen in the subsamples.

(iii) small n' may not learn all the data, which leads to underfitting, namely high bias and high variance.

large n' increases the correlation between different trees, which increases the variance and leads to overfitting

n' must be traded-off.

3 Gaussian Kernels

In this question, we will look at training a binary classifier with a Gaussian kernel. Given a set of labeled sample points $S = \{(X_i, y_i)\}_{i=1}^n \subseteq \mathbb{R}^d \times \{\pm 1\}$ and a kernel function $k(x, z)$, consider classifiers of the form

$$h(z) = \text{sign} \left(\sum_{i=1}^n a_i k(X_i, z) \right),$$

where $a \in \mathbb{R}^n$ is the vector of dual weights and $\text{sign}(u)$ is 1 if $u \geq 0$ or -1 if $u < 0$. To choose the n weights a_i , we solve the least-squares problem

$$a \in \arg \min_{a \in \mathbb{R}^n} \|Ka - y\|_2^2, \quad (1)$$

where $y \in \mathbb{R}^d$ is the vector of labels, with $y_i = \pm 1$, and K is the $n \times n$ kernel matrix with components $K_{ij} = k(X_i, X_j)$. Recall that the Gaussian kernel with bandwidth $\sigma > 0$ is

$$k(x, z) = \exp \left(-\frac{\|x - z\|_2^2}{2\sigma^2} \right).$$

- Consider a one-dimensional, two-point training set S (i.e., $d = 1$ and $n = 2$) with $(X_1, y_1) = (1, 1)$ and $(X_2, y_2) = (-1, -1)$. In the limit as the bandwidth parameter σ approaches zero, what is the optimal solution of a for the optimization problem (1) and what is the classifier $h(z)$?
- Now consider the classifier's behavior in the limit when the bandwidth parameter σ approaches infinity. Observe in this regime, the off-diagonal entries of the kernel matrix K approach one. Given an arbitrary training set S , suppose we solve the optimization problem (1) with all the components of K equal to one. Prove that if the number of $+1$ labels in S equals the number of -1 labels in S , then $a = \mathbf{0}$ is an optimal solution of (1). What is the resulting classifier $h(z)$ (in the simplest form possible)?
- Now consider the regime when the bandwidth parameter is large but finite. Consider again the two-point training set S with $(X_1, y_1) = (1, 1)$ and $(X_2, y_2) = (-1, -1)$. When $\sigma \gg 1$, we can approximate $k(X_1, X_2) \approx 1 + \frac{X_1 X_2}{2\sigma^2}$. Show that the solution of the optimization problem (1) with the kernel $k_a(X_1, X_2) = 1 + \frac{X_1 X_2}{2\sigma^2}$ is $a = (\sigma^2, -\sigma^2)$. What is the classifier $h(z)$?

Hint: By Cramer's Rule, the inverse of a 2×2 matrix is $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$.

$$(a) (X_1, y_1) = (1, 1) \quad (X_2, y_2) = (-1, -1)$$

$$a \in \arg \min \|Ka - y\|^2$$

$$\begin{aligned} 0 &= \nabla_a \|Ka - y\|^2 = \nabla_a (Ka - y)^T (Ka - y) = \nabla_a (a^T K^T - y^T) (Ka - y) \\ &= \nabla_a (y^T y + a^T K^T Ka - y^T Ka - a^T K^T y) \\ &= 2K^T Ka - 2K^T y \Rightarrow a = (K^T K)^{-1} K^T y \text{ is the optimal solution} \end{aligned}$$

$$\begin{aligned} K(X_1, X_2) &= K(X_2, X_1) = \exp \left(-\frac{2^2}{2\sigma^2} \right) = e^{-2/\sigma^2} \\ K(X_1, X_1) &= K(X_2, X_2) = \exp(-0) = 1 \end{aligned} \quad y = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\Rightarrow K = \begin{pmatrix} 1 & e^{-2/\sigma^2} \\ e^{-2/\sigma^2} & 1 \end{pmatrix}$$

$$a = \left[\begin{pmatrix} 1 & e^{-2/\sigma^2} \\ e^{-2/\sigma^2} & 1 \end{pmatrix} \begin{pmatrix} 1 & e^{-2/\sigma^2} \\ e^{-2/\sigma^2} & 1 \end{pmatrix} \right]^{-1} \begin{pmatrix} 1 & e^{-2/\sigma^2} \\ e^{-2/\sigma^2} & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$(K^T K)^{-1} K^T y = K^{-1} (K^T)^{-1} K^T y = K^{-1} y$$

$$a = \begin{pmatrix} 1 & e^{-2/\sigma^2} \\ e^{-2/\sigma^2} & 1 \end{pmatrix}^{-1} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} \frac{1}{1 - e^{-2/\sigma^2}} \\ \frac{-1}{1 - e^{-2/\sigma^2}} \end{pmatrix}$$

$$\sigma \rightarrow 0 \quad e^{-2/\sigma^2} \rightarrow e^{-\infty} \rightarrow 0$$

$$\Rightarrow a = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$h(z) = \text{sign} \left(\sum_{i=1}^n a_i K(x_i, z) \right) = \text{sign} \left(\exp \left(-\frac{(z-1)^2}{2\sigma^2} \right) - \exp \left(-\frac{(z+1)^2}{2\sigma^2} \right) \right)$$

(b) $K = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ is not invertible.

$$K^T K a = K^T y = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$K^T K = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$$

so $\vec{a} = 0$ namely $a_i = 0$ for all i .

$$h(z) = 0$$

$$K^T y = \begin{pmatrix} 1 & 1 & 1 & 1 \\ - & - & - & - \\ - & - & - & - \\ - & - & - & - \end{pmatrix}_{2n \times 2n} \begin{pmatrix} 1 \\ -1 \\ -1 \\ -1 \\ -1 \end{pmatrix}_{2n \times 1} = \vec{0}_{2n}$$

(c) $K(x_1, x_2) = 1 - \frac{1}{2\sigma^2}$

$$K(x_1, x_1) = K(x_2, x_2) = 1 + \frac{1}{2\sigma^2}$$

$$\Rightarrow K = \begin{pmatrix} 1 + \frac{1}{2\sigma^2} & 1 - \frac{1}{2\sigma^2} \\ 1 - \frac{1}{2\sigma^2} & 1 + \frac{1}{2\sigma^2} \end{pmatrix}$$

$$a = K^{-1} y = \frac{1}{4} \begin{pmatrix} 1+2\sigma^2 & 1-2\sigma^2 \\ 1-2\sigma^2 & 1+2\sigma^2 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$= \frac{1}{4} \begin{pmatrix} 4\sigma^2 \\ -4\sigma^2 \end{pmatrix} = \sigma^2 \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$h(z) = \text{sign} \left(a_1 K(x_1, z) + a_2 K(x_2, z) \right)$$

$$= \sigma^2 \text{sign} \left(\left(1 + \frac{z}{2\sigma^2} \right) - \left(1 - \frac{z}{2\sigma^2} \right) \right)$$

$$= \sigma^2 \text{sign} \left(\frac{z}{\sigma^2} \right) = \text{sign}(z)$$

4 Decision Trees for Classification

In this problem, you will implement decision trees and random forests for classification on two datasets: 1) the spam dataset and 2) a Titanic dataset to predict survivors of the infamous disaster. The data is with the assignment. See the Appendix for more information on its contents and some suggestions on data structure design.

In lectures, you were given a basic introduction to decision trees and how such trees are trained. You were also introduced to random forests. Feel free to research different decision tree techniques online. You do not have to implement boosting (which we will learn late this semester), although it might help with Kaggle.

For your convenience, we provide starter code which includes preprocessing and some decision tree functionality already implemented. Feel free to use (or not to use) this code in your implementation.

4.1 Implement Decision Trees

We expect you to implement the tree data structure yourself; you are not allowed to use a pre-existing decision tree implementation. The Titanic dataset is not “cleaned”—that is, there are missing values—so you can use external libraries for data preprocessing and tree visualization (in fact, we recommend it). Removing examples with missing features is not a good option; there is not enough data to justify throwing some of it away. Be aware that some of the later questions might require special functionality that you need to implement (e.g., maximum depth stopping criterion, visualizing the tree, tracing the path of a sample point through the tree). You can use any programming language you wish as long as we can read and run your code with minimal effort. If you choose to use our starter code, a skeleton structure of the decision tree implementation is provided, and you will decide how to fill it in. In this part of your writeup, **include your decision tree code**.

4.2 Implement a Random Forest

You are not allowed to use any off-the-shelf random forest implementation. If you architected your code well, this part should be a (relatively) easy encapsulation of the previous part. In this part of your writeup, **include your random forest code**.

4.3 Describe implementation details

We aren’t looking for an essay; 1–2 sentences per question is enough.

1. How did you deal with categorical features and missing values?
2. What was your stopping criterion?
3. How did you implement random forests?
4. Did you do anything special to speed up training?
5. Anything else cool you implemented?

4.4 Performance Evaluation

For each of the 2 datasets, train both a decision tree and random forest and report your training and validation accuracies. You should be reporting 8 numbers (2 datasets $\times$ 2 classifiers $\times$ training/validation). In addition,

for both datasets, train your best model and submit your predictions to Kaggle. Include your Kaggle display name and your public scores on each dataset. You should be reporting 2 Kaggle scores.

4.5 Writeup Requirements for the Spam Dataset

1. (Optional) If you use any other features or feature transformations, explain what you did in your report. You may choose to use something like bag-of-words. You can implement any custom feature extraction code in `featurize.py`, which will save your features to a `.mat` file.
2. For your decision tree, and for a data point of your choosing from each class (spam and ham), state the splits (i.e., which feature and which value of that feature to split on) your decision tree made to classify it. An example of what this might look like:
 - (a) (“viagra”) ≥ 2
 - (b) (“thanks”) < 1
 - (c) (“nigeria”) ≥ 3
 - (d) Therefore this email was spam.
 - (a) (“budget”) ≥ 2
 - (b) (“spreadsheet”) ≥ 1
 - (c) Therefore this email was ham.
3. Generate a random 80/20 training/validation split. Train decision trees with varying maximum depths (try going from depth = 1 to depth = 40) with all other hyperparameters fixed. Plot your validation accuracies as a function of the depth. Which depth had the highest validation accuracy? Write 1–2 sentences explaining the behavior you observe in your plot. If you find that you need to plot more depths, feel free to do so.

4.6 Writeup Requirements for the Titanic Dataset

Train a very shallow decision tree (for example, a depth 3 tree, although you may choose any depth that looks good) and visualize your tree. Include for each non-leaf node the feature name and the split rule, and include for leaf nodes the class your decision tree would assign. You can use any visualization method you want, from simple printing to an external library; the `rcviz` library on github works well.

A Appendix

Data Processing for Titanic

Here's a brief overview of the fields in the Titanic dataset. You will need to preprocess the dataset into a form usable by your decision tree code.

1. survived: the label we want to predict. 1 indicates the person survived, whereas 0 indicates the person died.
2. pclass: Measure of socioeconomic status. 1 is upper, 2 is middle, 3 is lower.
3. age: Fractional if less than 1.
4. sex: Male/female.
5. sibsp: Number of siblings/spouses aboard the Titanic.
6. parch: Number of parents/children aboard the Titanic.
7. ticket: Ticket number.
8. fare: Fare.
9. cabin: Cabin number.
10. embarked: Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

You will face two challenges you did not have to deal with in previous datasets:

1. Categorical variables. Most of the data you've dealt with so far has been continuous-valued. Some features in this dataset represent types/categories. Here are two possible ways to deal with categorical variables:
 - (a) (Easy) In the feature extraction phase, map categories to binary variables. For example suppose feature 2 takes on three possible values: 'TA', 'lecturer', and 'professor'. In the data matrix, these categories would be mapped to three binary variables. These would be columns 2, 3, and 4 of the data matrix. Column 2 would be a boolean feature $\{0, 1\}$ representing the TA category, and so on. In other words, 'TA' is represented by $[1, 0, 0]$, 'lecturer' is represented by $[0, 1, 0]$, and 'professor' is represented by $[0, 0, 1]$. Note that this expands the number of columns in your data matrix. This is called "vectorizing," or "one-hot encoding" the categorical feature.
 - (b) (Hard, but more generalizable) Keep the categories as strings or map the categories to indices (e.g. 'TA', 'lecturer', 'professor' get mapped to 0, 1, 2). Then implement functionality in decision trees to determine split rules based on the subsets of categorical variables that maximize information gain. You cannot treat these as normal continuous-valued features because ordering has no meaning for these categories (the fact that $0 < 1 < 2$ has no significance when 0, 1, 2 are discrete categories).
2. Missing values. Some data points are missing features. In the csv files, these are represented by the value '?'. You have three approaches:

- (a) (Easiest) If a data point is missing some features, remove it from the data matrix (**this is useful for your first code draft, but your submission must not do this**).
- (b) (Easy) Infer the value of the feature from all the other values of that feature (e.g., fill it in with the mean, median, or mode of the feature. Think about which of these is the best choice and why).
- (c) (Hard, but more powerful). Use k -nearest neighbors to impute feature values based on the nearest neighbors of a data point. In your distance metric you will need to define the distance to a missing value.
- (d) (Hardest, but more powerful) Implement within your decision tree functionality to handle missing feature values based on the current node. There are many ways this can be done. You might infer missing values based on the mean/median/mode of the feature values of data points sorted to the current node. Another possibility is assigning probabilities to each possible value of the missing feature, then sorting fractional (weighted) data points to each child (you would need to associate each data point with a weight in the tree).

For Python:

It is recommended you use the following classes to write, read, and process data:

```
csv.DictReader
sklearn.feature_extraction.DictVectorizer (vectorizing categorical variables)
    (There's also sklearn.preprocessing.OneHotEncoder, but it's much less clean)
sklearn.preprocessing.LabelEncoder
    (if you choose to discretize but not vectorize categorical variables)
sklearn.preprocessing.Imputer
    (for inferring missing feature values in the preprocessing phase)
```

If you use `csv.DictReader`, it will automatically parse out the header line in the csv file (first line of the file) and assign values to fields in a dictionary. This can then be consumed by `DictVectorizer` to binarize categorical variables.

To speed up your work, you might want to store your cleaned features in a file, so that you don't need to preprocess every time you run your code.

Suggested Architecture

This is a complicated coding project. You should put in some thought about how to structure your program so your decision trees don't end up as horrific forest fires of technical debt. Here is a rough, **optional** spec that only covers the barebones decision tree structure. This is only for your benefit—writing clean code will make your life easier, but we won't grade you on it. There are many different ways to implement this.

Your decision trees ideally should have a well-encapsulated interface like this:

```
classifier = DecisionTree(params)
classifier.train(train_data, train_labels)
predictions = classifier.predict(test_data)
```

where `train_data` and `test_data` are 2D matrices (rows are data, columns are features).

A decision tree (or **DecisionTree**) is a binary tree composed of **Nodes**. You first initialize it with the necessary parameters (which depend on what techniques you implement). As you train your tree, your tree should create and configure **Nodes** to use for classification and store these nodes internally. Your **DecisionTree** will store the root node of the resulting tree so you can use it in classification.

Each **Node** has left and right pointers to its children, which are also nodes, though some (like leaf nodes) won't have any children. Each node has a split rule that, during classification, tells you when you should continue traversing to the left or to the right child of the node. Leaf nodes, instead of containing a split rule, should simply contain a label of what class to classify a data point as. Leaf nodes can either be a special configuration of regular **Nodes** or an entirely different class.

Node fields:

- **split_rule**: A length 2 tuple that details what feature to split on at a node, as well as the threshold value at which you should split. The former can be encoded as an integer index into your data point's feature vector.
- **left**: The left child of the current node.
- **right**: The right child of the current node.
- **label**: If this field is set, the **Node** is a leaf node, and the field contains the label with which you should classify a data point as, assuming you reached this node during your classification tree traversal. Typically, the label is the mode of the labels of the training data points arriving at this node.

DecisionTree methods:

- **entropy(labels)**: A method that takes in the labels of data stored at a node and compute the entropy for the distribution of the labels.
- **information_gain(features, labels, threshold)**: A method that takes in some feature of the data, the labels and a threshold, and compute the information gain of a split using the threshold.
- **entropy(label)**: A method that takes in the labels of data stored at a node and compute the entropy (or Gini impurity).
- **fit(data, labels)**: Grows a decision tree by constructing nodes. Using the entropy and segmenter methods, it attempts to find a configuration of nodes that best splits the input data. This function figures out the split rules that each node should have and figures out when to stop growing the tree and insert a leaf node. There are many ways to implement this, but eventually your **DecisionTree** should store the root node of the resulting tree so you can use the tree for classification later on. Since the height of your **DecisionTree** shouldn't be astronomically large (you may want to cap the height—if you do, the max height would be a hyperparameter), this method is best implemented recursively.
- **predict(data)**: Given a data point, traverse the tree to find the best label to classify the data point as. Start at the root node you stored and evaluate split rules at each node as you traverse until you reach a leaf node, then choose that leaf node's label as your output label.

Random forests can be implemented without code duplication by storing groups of decision trees. You will have to train each tree on different subsets of the data (data bagging) and train nodes in each tree on different subsets of features (attribute bagging). Most of this functionality should be handled by a random forest class, except attribute bagging, which may need to be implemented in the decision tree class. Hopefully, the

spec above gives you a good jumping-off point as you start to implement your decision trees. Again, it's highly recommended to think through design before coding.

Happy hacking!

Submission Checklist

Please ensure you have completed the following before your final submission.

At the beginning of your writeup...

1. Have you copied and hand-signed the honor code specified in Question 1?
2. Have you listed all students (Names and ID numbers) that you collaborated with?

In your writeup for Question 4...

1. Have you included your **Kaggle Score** and **Kaggle Username**?
2. Have you included your generated plots and visualizations?

At the end of the writeup...

1. Have you provided a code appendix including all code you wrote in solving the homework?

Executable Code Submission

1. Have you created an archive containing all “.py” files that you wrote or modified to generate your homework solutions?
2. Have you removed all data and extraneous files from the archive?
3. Have you included a README in your archive containing any special instructions to reproduce your results?

Submissions

1. Have you submitted your written solutions to the Gradescope assignment titled **HW5 Write-Up** and selected pages appropriately?
2. Have you submitted your executable code archive to the Gradescope assignment titled **HW5 Code**?
3. Have you submitted your test set predictions for **Spam** and **Titanic** dataset to the appropriate Kaggle challenges?
4. Is your Kaggle submission in integer format? Submissions in decimal format will receive a score of zero!

Congratulations! You have completed Homework 5.